

MoE 架构详解：从稀疏路由到 Expert Parallel

原始问题：详细讲解 MoE 架构

提问：2026/07/29 20:27 更新：2026/07/29 20:27 重要度：5/5

MoE 架构详解：从稀疏路由到 Expert Parallel

一句话结论

MoE (Mixture of Experts, 混合专家) 是在 Transformer 的 FFN 位置放置多个独立 MLP Experts, 再让一个可学习 Router 为每个 Token 只选择少数 Top-k Experts。

它的核心价值是：

总参数量由全部 E 个 Experts 决定
每 Token 计算量只由激活的 k 个 Experts 决定
且通常 $k \ll E$

因此 MoE 可以让模型拥有非常大的参数容量，而不让每个 Token 都执行全部参数。

但 MoE 不是“免费扩大模型”：

- 所有 Expert 参数仍要存储、优化和加载；
- Router 可能把大量 Token 挤到少数 Experts，产生负载不均；
- Expert Parallel 需要两次 Token All-to-All；
- 每个 Expert 收到的 Token 数动态变化，容易形成小 GEMM 和 GPU 低利用率；
- 训练需要处理负载均衡、容量溢出、Token Drop 和 Router 稳定性；
- 推理解码阶段 Batch 小、Token 少，MoE 的通信和专家权重访问更难摊薄。

一句话记忆：

Dense 模型让所有 Token 使用同一组 FFN 参数；MoE 让不同 Token 动态选择不同 FFN 参数。它用“条件计算”换取“更大的参数容量”。

一、MoE 在 Transformer 中放在哪里

标准 Transformer Block 通常包含：

```
Input
→ Attention
→ Residual
→ Dense FFN / MLP
→ Residual
→ Output
```

MoE 通常不替换 Attention，而是把某些层的 Dense FFN 替换为 Sparse MoE FFN：

```
Input
→ Attention
→ Residual
→ Router + Experts + Dispatch/Combine
→ Residual
→ Output
```

Dense FFN 对所有 Token 使用同一套参数：

$$\text{FFN}(x) = W_2 \cdot \text{activation}(W_1 x)$$

MoE 对 Token x_t 先选择专家集合 S_t ：

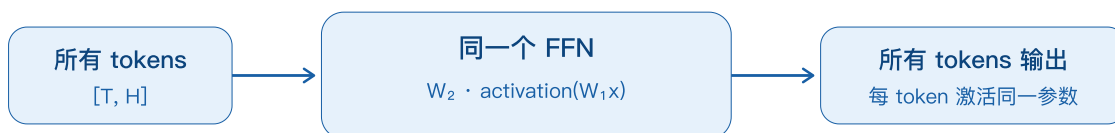
$$\text{MoE}(x_t) = \sum_{e \in S_t} g_{\{t,e\}} \cdot \text{Expert}_e(x_t)$$

其中：

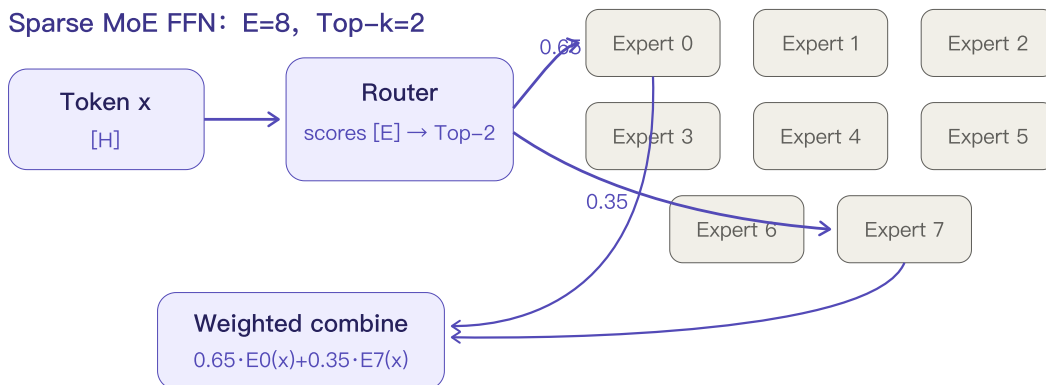
E	Expert 总数
k	每个 Token 激活的 Expert 数
S_t	Token t 选中的 Top-k Expert 集合
$g_{\{t,e\}}$	Token t 对 Expert e 的路由权重

Transformer block 中: Attention 不变, FFN 被稀疏专家层替换

Dense FFN



Sparse MoE FFN: E=8, Top-k=2



总参数可含 8 个专家, 但该 token 只执行 2 个专家: 参数容量与每 token 计算量解耦。

MoE 不是什么

- 不是 Ensemble: 不是把多个完整模型都跑一遍再投票;
- 不是 Data Parallel: 不同 Experts 是不同参数, 不是同一模型副本;
- 不是简单模型分片: Router 会按 Token 动态决定执行路径;
- 不是减少总参数显存: 总 Expert 参数可能远大于 Dense 模型;
- 不是所有层都必须 MoE: 常见架构会按固定频率插入 MoE 层, 其他层仍为 Dense。

二、统一例子: 全文共用的 Shape 与参数

为了让结构、公式和代码完全对应, 使用一个小型可手算例子:

```
Batch B = 2
Sequence S = 3
Token 数 T = B × S = 6
Hidden Size H = 8
Expert 总数 E = 4
Top-k = 2
每个 Expert 的中间维度 F_expert = 16
对照 Dense FFN 的中间维度 F_dense = 32
Dtype = float32 (代码示例)
```

输入 Hidden States:

```
X: [B, S, H] = [2, 3, 8]
Flatten 后: X_flat: [T, H] = [6, 8]
```

Router 权重:

```
W_r: [H, E] = [8, 4]
Router Logits: X_flat @ W_r = [6, 4]
```

每个 Token 选择 2 个 Experts:

```
Top-k Indices: [T, k] = [6, 2]
Top-k Weights: [T, k] = [6, 2]
```

总 Token-Expert Assignments:

```
A = T × k = 6 × 2 = 12
```

为什么设置 `F_dense=32`、`F_expert=16`?

简单两层 MLP 每 Token FLOP 近似与中间维度成正比。Top-2 会执行两个 `F=16` 的 Experts:

```
2 × 16 = 32
```

因此它与一个 `F_dense=32` 的 Dense FFN 具有接近的主 MLP 计算量，便于比较参数容量与激活计算。

三、Dense FFN 与 MoE 的参数量、激活参数和 FLOP

先使用最简单的两层 MLP，忽略 Bias:

```
Expert(x) = W2 · activation(W1x)
W1: [H, F]
W2: [F, H]
```

3.1 Dense FFN 参数量

```
P_dense = H×F_dense + F_dense×H
          = 2HF_dense
```

代入：

```
P_dense = 2 × 8 × 32  
         = 512 params
```

3.2 MoE 全部 Expert 参数量

单个 Expert：

```
P_one_expert = 2HF_expert  
              = 2 × 8 × 16  
              = 256 params
```

4 个 Experts：

```
P_all_experts = E × 2HF_expert  
               = 4 × 256  
               = 1024 params
```

Router：

```
P_router = H × E  
          = 8 × 4  
          = 32 params
```

MoE 总参数：

```
P_moe_total = 1024 + 32 = 1056 params
```

3.3 每 Token 激活的参数量

Top-2 每 Token 只执行两个 Experts：

```
P_active_experts_per_token  
= k × 2HF_expert  
= 2 × 256  
= 512 params
```

加上 Router 投影约 32 个参数参与计算：

```
P_active_per_token ≈ 544 params
```

因此这个例子中：

指标	Dense FFN	MoE FFN
总参数	512	1056
每 Token 主 MLP 激活参数	512	512
Router 参数	0	32
Expert 数	1	4
每 Token 激活 Experts	1	2

结论：主 MLP 计算近似不变，但总参数容量约扩大 2 倍。

如果保持每个 Expert 和 Dense FFN 同样宽，同时使用 Top-2，那么每 Token Expert 计算会约变成 Dense FFN 的 2 倍。MoE 是否“同计算量”取决于：

$k \times F_{\text{expert}}$ 是否接近 F_{dense}

不能只看 $k \ll E$ 就断言计算完全不变。

3.4 FLOP 估算

矩阵乘的 Multiply-Add 按 2 FLOP 计。两层简单 MLP 每 Token：

$\text{FLOP}_{\text{one_expert}} \approx 4\text{HF}_{\text{expert}}$

Top-k：

$\text{FLOP}_{\text{moe_experts_per_token}} \approx 4k\text{HF}_{\text{expert}}$

Router：

$\text{FLOP}_{\text{router_per_token}} \approx 2\text{HE}$

代入：

Expert FLOP $\approx 4 \times 2 \times 8 \times 16 = 1024$ FLOP/token
Router FLOP $\approx 2 \times 8 \times 4 = 64$ FLOP/token

Dense 对照：

$\text{FLOP}_{\text{dense}} \approx 4 \times 8 \times 32 = 1024$ FLOP/token

因此：

主 Expert MLP FLOP $\approx$ Dense MLP FLOP
MoE 额外增加 Router、Permutation 和通信成本

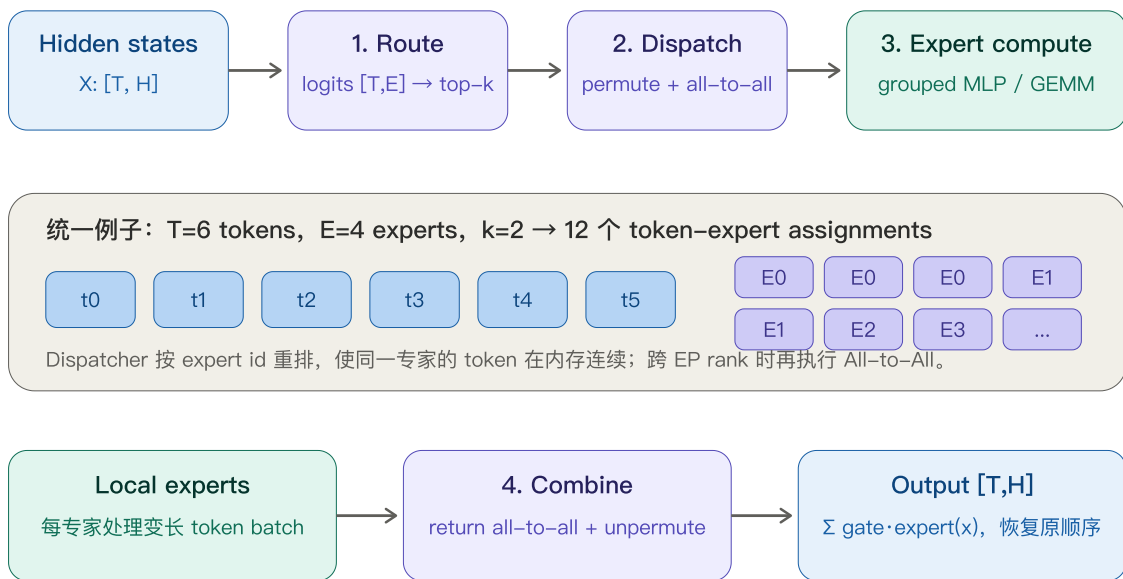
现代 Transformer 常使用 SwiGLU/Gated MLP，包含 3 个线性投影，参数量和 FLOP 系数会从 **2HF/4HF** 变为大约 **3HF/6HF**；但“总参数按 E 增长、激活计算按 k 增长”的关系不变。

四、MoE Layer 的四个阶段

一个 MoE Layer 的完整 Forward 可拆成：

1. Route
2. Dispatch
3. Expert Compute
4. Combine

一个 MoE FFN 的完整前向：Route $\rightarrow$ Dispatch $\rightarrow$ Compute $\rightarrow$ Combine



动态路由使每个专家收到的 token 数不同：负载不均、容量溢出和小 GEMM 是 MoE 的核心系统难点。

4.1 Route：为每个 Token 选择 Experts

输入：

$x: [T, H]$

Router 是一个小型线性投影：

```
L = XW_r

X:   [T, H]
W_r: [H, E]
L:   [T, E]
```

将 Logits 转成 Scores。经典形式：

```
P = softmax(L, dim=-1)
```

然后选择 Top-k：

```
indices, weights = topk(P, k)
```

通常会对选中的 **k** 个权重重新归一化：

$$g_{\{t,e\}} = p_{\{t,e\}} \div \sum_{\{j \in S_t\}} p_{\{t,j\}}$$

于是：

$$\sum_{\{e \in S_t\}} g_{\{t,e\}} = 1$$

部分架构使用 Sigmoid Score、Group-Limited Routing、Expert Bias 或 Aux-Loss-Free Balancing；不能把所有 MoE 都等同于“全局 Softmax + Top-2”。

4.2 Dispatch：把 Token 发给目标 Experts

Router 产生的是 Token→Expert Assignment。Expert 计算要求同一个 Expert 的 Token 在内存中连续，因此先 Permute：

```
原顺序: t0, t1, t2, t3, t4, t5
按 Expert 分组后:
E0 的 tokens | E1 的 tokens | E2 的 tokens | E3 的 tokens
```

如果所有 Experts 在同一 GPU，只需本地重排。

如果 Experts 分布在不同 GPU，需要执行 Expert Parallel 通信：

```
Dispatch All-to-All
```

把每个 Token Assignment 发送给目标 Expert 所在的 Rank。

4.3 Expert Compute: 每个 Expert 执行 MLP

Expert `e` 收到:

```
X_e: [T_e, H]
```

其中 `T_e` 是动态的, 每个 Expert 不同。

执行:

```
Y_e = Expert_e(X_e)
```

由于 `T_e` 可能很小、差异很大, 如果逐 Expert 启动独立 GEMM, 会产生:

- 大量小 GEMM;
- Kernel Launch 开销;
- Tensor Core 利用率低;
- 最慢 Expert/Rank 决定整体延迟。

因此生产实现常使用 Grouped GEMM: 把多个不同 `T_e` 的 Expert GEMM 组织在一次或少量 Kernel 中执行。

4.4 Combine: 返回原 Rank, 恢复顺序并加权

Expert 输出先通过第二次通信返回 Token 原始 Rank:

```
Combine All-to-All
```

然后 Unpermute 恢复原 Token 顺序, 并按 Router 权重合并:

```
y_t = \sum_{e \in S_t} g_{\{t,e\}} \cdot \text{Expert}_e(x_t)
```

最终输出 Shape 恢复为:

```
Y: [T,H] \rightarrow [B,S,H]
```

五、手算一个完整 Top-2 Routing 例子

设 6 个 Token 对 4 个 Experts 的 Top-2 结果如下:

Token	Expert 1	Weight 1	Expert 2	Weight 2
t0	E0	0.65	E3	0.35
t1	E1	0.70	E0	0.30
t2	E2	0.60	E3	0.40
t3	E0	0.55	E2	0.45
t4	E3	0.80	E1	0.20
t5	E1	0.52	E2	0.48

每 Expert Token Assignment 数：

```
E0: t0, t1, t3 → 3
E1: t1, t4, t5 → 3
E2: t2, t3, t5 → 3
E3: t0, t2, t4 → 3
```

总 Assignment：

```
3 + 3 + 3 + 3 = 12 = T×k
```

Token **t0** 输出：

```
y_0 = 0.65·Expert_0(x_0) + 0.35·Expert_3(x_0)
```

同一个 Token 被复制成两个 Expert Assignments，但最终仍合并回一个 **[H]** 输出。

这个例子完全均衡。真实 Router 很容易出现：

```
E0: 6 assignments
E1: 3
E2: 2
E3: 1
```

此时 E0 变成热点，其他 Experts 空闲。

六、Expert Capacity、Capacity Factor 与 Token Dropping

6.1 平均容量

总 Assignment 数：

$$A = T \times k$$

均匀分给 E 个 Experts，每 Expert 平均收到：

$$A_{avg} = T \times k \div E$$

容量通常定义为：

$$\begin{aligned} \text{Capacity } C \\ = \text{ceil}(T \times k \times \text{CapacityFactor} \div E) \end{aligned}$$

Megatron Core 文档给出的同类公式以每 Rank Token 数为口径：

$$\begin{aligned} \text{capacity} \\ = \text{num_tokens_per_rank} \times \text{topk} \times \text{capacity_factor} \div \text{num_experts} \end{aligned}$$

具体是全局还是局部 Token 口径、是否向上取整、容量发生在通信前还是后，要以框架实现为准。

统一例子：

$$\begin{aligned} T=6, k=2, E=4, \text{CapacityFactor}=1.0 \\ C=\text{ceil}(6 \times 2 \div 4)=3 \end{aligned}$$

均衡路由时每 Expert 正好 3 个 Assignments，不溢出。

若 $E=6$ 收到 6 个：

$$\text{overflow} = 6 - 3 = 3$$

6.2 Token Dropping

Capacity-Based MoE 会只保留容量内的 Token Assignments，超出的 Assignment 被 Drop，常按 Router Probability 优先保留高分项。

风险：

- 被 Drop 的 Expert 路径不执行；
- 若 Token 的所有路径都被 Drop，会造成信息损失；
- Capacity 太小会影响模型质量；
- Capacity 太大则增加 Padding、显存和通信。

6.3 Padding to Capacity

为了得到固定 GEMM Shape，可以把不足容量的 Expert 输入补齐到 **C**：

```
Expert 真实收到 1 个 Token  
Capacity = 3  
→ Padding 到 3
```

优点：

- Shape 固定；
- Kernel 更规则；
- CUDA Graph 更容易；
- 集合通信尺寸可预测。

缺点：

- 计算 Padding；
- 浪费显存与带宽；
- Capacity Factor 越大，浪费越明显。

6.4 Dropless MoE

Dropless 实现不丢 Token，也不强制全部 Padding 到统一容量。每个 Expert 处理真实动态 Token 数。

优点：保留所有路由信息。

代价：

- 动态 Shape；
- Expert Token 数高度不均；
- Grouped GEMM 和通信实现更复杂；
- CUDA Graph、内存规划和负载均衡更难。

七、为什么 Router 会塌缩：负载均衡问题

如果某个 Expert 在训练早期偶然更擅长一些 Token，Router 会给它更高分；它获得更多数据，更新更充分，于是进一步变强：

```
更高路由概率
→ 更多 Token
→ 更多梯度和训练机会
→ Expert 更强
→ 更高路由概率
```

这会形成正反馈，导致：

- 少数 Experts 过载；
- 其他 Experts 几乎没有训练数据；
- All-to-All Rank 不均衡；
- 最慢 Rank 拖住全组；
- 容量溢出和 Token Drop 增加；
- Expert Specialization 退化。

八、Load Balancing Auxiliary Loss

一种经典思路同时约束：

1. 每个 Expert 实际分到的 Assignment 比例；
2. Router 给每个 Expert 的平均概率质量。

定义：

```
f_e = Expert e 实际接收的 Assignment 数 ÷ (T×k)
p_e = (1/T) × Σ_t P_{t,e}
```

一个常见形式：

```
L_aux = α × E × Σ_e f_e p_e
```

其中：

```
α Auxiliary Loss 系数
```

若完全均匀：

```
f_e = 1/E
p_e = 1/E

E × Σ_e (1/E × 1/E)
= E × E × 1/E²
= 1
```

训练时通常把它加到主 Loss：

```
L_total = L_task + L_aux + L_z + ...
```

不同论文和框架对 Top-k、归一化和比例系数的具体定义可能不同，不能看到“Aux Loss”就假设公式完全一致。

Aux Loss 的副作用

Aux Loss 太小：负载不均控制不住。

Aux Loss 太大：Router 为了均匀而均匀，可能损害语义路由与 Expert Specialization。

所以负载均衡与专家专门化存在张力：

```
绝对均匀不一定是质量最优
严重不均一定是系统灾难
```

现代系统还会使用：

- Expert Bias 动态调节；
- Aux-Loss-Free Balancing；
- Group-Limited Routing；
- Node-Limited Routing；
- Sinkhorn/S-BASE 等方法。

九、Router Z-Loss 与数值稳定性

Router Logits 可能持续增大，使 Softmax 饱和、梯度和低精度训练不稳定。

常见 Router Z-Loss：

$$L_z = \lambda_z \times \text{mean}_t[(\log \sum_e \exp(l_{t,e}))^2]$$

它鼓励 Logits 不要无界增大。

还可能使用：

- Router 使用 FP32 计算；
- Input Jitter/Noise；
- Router Weight 初始化控制；
- Logit Clipping；
- 更稳定的 Softmax/Fused Kernel。

Router 参数很少，但它决定每个 Token 走哪条计算路径，因此数值问题可能放大成整个系统的路由异常。

十、Expert Parallelism: Experts 如何分到多张 GPU

当 Expert 总参数无法或不应复制到每张 GPU 时，使用 Expert Parallel (EP)。

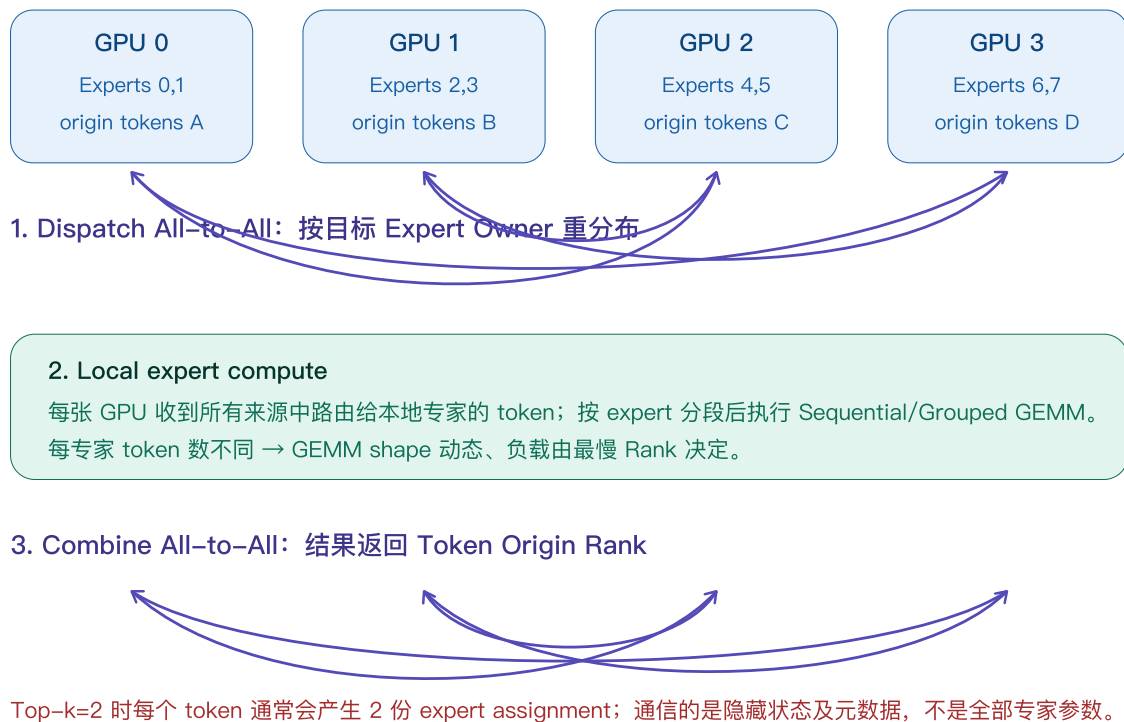
统一并行例子：

```
E = 8 Experts
EP = 4 GPUs
每 GPU Local Experts = E ÷ EP = 2
```

映射：

```
GPU 0: Experts 0,1
GPU 1: Experts 2,3
GPU 2: Experts 4,5
GPU 3: Experts 6,7
```

EP=4, E=8: 每张 GPU 持有 2 个专家



一个 Rank 上的本地 Token 可能被 Router 选到任意 Expert。因此 Forward 需要:

```
本地 Tokens
→ Router
→ 按 Expert Owner 分组
→ Dispatch All-to-All
→ Local Expert Compute
→ Combine All-to-All
→ 恢复原 Token 顺序
```

10.1 第一次 All-to-All: Dispatch

每个 Rank 把 Token Hidden State 发送到目标 Expert 所在 Rank。

发送的是:

- Hidden State;
- 路由权重;
- Token/Expert 索引或恢复顺序所需元数据。

不是把全部 Expert 参数来回发送。

10.2 第二次 All-to-All: Combine

Expert 计算完成后, 结果返回 Token 原始 Rank, 再进行 Unpermute 与 Weighted Sum。

Backward 会沿相反的数据依赖执行对应的通信与梯度计算，因此训练通信量比 Forward-only 更大。

十一、All-to-All 通信量手算

设某 EP Rank 原始拥有：

```
T_local = 4096 Tokens
H = 4096
Dtype = BF16 = 2 Byte
k = 2
EP = 8
```

总 Token Assignments：

```
A_local = T_local × k
          = 4096 × 2
          = 8192
```

若路由均衡，目标 Expert 在本 Rank 的概率约为 $1/EP$ ，需要发往远端的比例约：

```
(EP-1)/EP = 7/8
```

一次 Dispatch 的远端 Hidden State 发送量近似：

```
Bytes_dispatch_send
≈ T_local × k × H × dtype_bytes × (EP-1)/EP

= 4096 × 2 × 4096 × 2 × 7/8
= 58,720,256 Byte
= 56 MiB
```

Combine 方向近似再发送一次：

```
Forward 两次 All-to-All 发送 Payload
≈ 56 MiB + 56 MiB
= 112 MiB / Rank / MoE Layer
```

这还没有包括：

- 元数据；
- 对齐/Padding；
- 接收流量；
- Backward 通信；

- 不均衡导致的额外等待；
- 网络协议与实际带宽折损。

因此 MoE 常出现：参数 FLOP 看起来很省，但 All-to-All 却成为 Step Time 的主要瓶颈。

十二、为什么 All-to-All 很难优化

AllReduce 中每个 Rank 的消息量和形状通常可预测；MoE All-to-All 的消息量由本批 Token 路由结果动态决定：

Rank 0 可能向 Rank 1 发很多 Token
Rank 2 可能几乎不向 Rank 1 发
下一批又完全不同

难点包括：

- 每个 Peer 消息尺寸不同；
- 网络热点随 Batch 变化；
- 最慢 Rank 决定全组完成时间；
- 跨节点带宽远低于机内 NVLink；
- Token Permutation/Unpermutation 也消耗 HBM 带宽；
- 通信和 Expert GEMM 可能同时争用 HBM；
- Top-k 越大，Assignments 与通信近似线性增加。

常见优化：

- 尽量把 EP Group 放在高速互联域；
- 使用优化 Dispatcher，如 All-to-All/Flex/DeepEP 类后端；
- Token Permutation Fusion；
- Dispatch/Combine 与 Shared Expert 或其他计算 Overlap；
- Node-Limited Routing，减少跨节点目的地；
- 低精度通信；
- 调整 EP、TP、DP、CP 的映射；
- Batch-Level Overlap。

十三、MoE 能否与计算 Overlap

可以，但比 DDP AllReduce Overlap 更复杂。

一个典型机会：

```
Routed Experts 路径: Route → Dispatch A2A → Expert Compute → Combine A2A
Shared Expert 路径: Shared Expert MLP
```

如果两条路径没有直接依赖，可以：

```
Dispatch All-to-All 与 Shared Expert Compute 重叠
Expert Compute 与其他通信阶段流水化
Combine 与后续可独立任务重叠
```

也可以把 Token 分 Chunk，形成：

```
Chunk 0 通信
→ Chunk 0 计算, 同时 Chunk 1 通信
→ Chunk 0 返回, 同时 Chunk 1 计算
```

但实际 Overlap 受限于：

- NCCL/通信 Kernel 是否占 SM；
- Expert GEMM 和通信是否争抢 HBM；
- 消息是否足够大；
- Chunk 太小导致 Launch 开销；
- 路由结果是否提前可用；
- CUDA Stream/Event 依赖；
- 跨节点带宽与拓扑。

因此“时间线上重叠”不等于“通信完全隐藏”。要用 Nsight Systems 和框架 Timer 看：

```
exposed dispatch time
exposed combine time
expert GEMM utilization
per-expert token histogram
per-rank send/recv bytes
```

十四、Expert Compute 为什么需要 Grouped GEMM

假设本地有 4 个 Experts, Token 数分别为:

```
T_0=128, T_1=64, T_2=19, T_3=173
```

若逐个执行:

```
GEMM Expert 0  
GEMM Expert 1  
GEMM Expert 2  
GEMM Expert 3
```

问题:

- 4 次或更多 Kernel Launch;
- 小 $M=T_e$ GEMM 难以占满 GPU;
- 动态 Shape 多;
- Expert 2 只有 19 个 Token, 效率很低。

Grouped GEMM 把多个不同 M 、相同或兼容 K/N 的 GEMM 组织起来:

```
一次 Grouped GEMM  
处理多个 Expert 的 Token 分段和权重分段
```

收益:

- 减少 Launch;
- 更好地调度 Tensor Core 工作;
- 提升多个小 Expert 的整体利用率;
- 与 FP8/低精度 Expert Kernel 结合。

但 Grouped GEMM 不能消除负载不均: 如果某个 Rank 收到远多于其他 Rank 的 Token, 其他 Rank 仍要等待。

十五、Shared Experts 与 Routed Experts

传统 MoE 所有知识都由 Routed Experts 承担, 可能产生:

- 不同 Experts 重复学习通用知识；
- Router 为常见模式浪费路由容量；
- Routed Expert Specialization 不够纯粹。

Shared Expert 对所有 Token 始终执行：

$$Y = \text{SharedExpert}(x) + \sum_{\{e \in S_t\}} g_{\{t, e\}} \text{RoutedExpert}_e(x)$$

作用：

- Shared Expert 学习通用知识；
- Routed Experts 更专注差异化知识；
- 减少 Routed Experts 之间的冗余；
- Shared 路径可作为通信 Overlap 的计算来源。

代价：

- Shared Expert 是 Dense Compute，所有 Token 都执行；
- 增加每 Token FLOP；
- 需要设计共享与路由输出如何融合；
- 可能再次成为计算瓶颈。

十六、Fine-Grained Expert Segmentation

DeepSeekMoE 提出的核心思想之一是把较大的 Experts 细分为更多、更小的 Experts。

传统：

N 个大 Experts，激活 K 个

细粒度方案：

mN 个小 Experts，激活 mK 个

若每个小 Expert 大约是原 Expert 的 $\frac{1}{m}$ 宽度，则激活总计算可保持接近，但组合方式更多：

$C(N, K)$ 种粗粒度组合
 $\rightarrow C(mN, mK)$ 种更灵活的细粒度组合

目标：让专家知识分解得更细，Token 可以组合多个更专门的能力单元。

代价：

- Experts 更小，单个 GEMM 更碎；
- Router 输出维度更大；
- Assignment 数更多；
- Dispatch 元数据和通信更复杂；
- 更依赖 Grouped GEMM 和高效 Dispatcher。

因此 Fine-Grained Experts 是模型表达能力和系统效率的共同设计，而不是单纯增加 Expert 数。

十七、训练阶段与推理阶段有什么不同

17.1 训练

训练需要：

- Router 梯度；
- Expert 参数梯度；
- Load Balancing Aux Loss；
- Router Z-Loss；
- Forward 与 Backward All-to-All；
- Optimizer State；
- Expert 参数分片/复制；
- Activation 保存或重计算。

训练的 Token 数通常较大，Grouped GEMM 和通信更容易形成大批量，但 All-to-All 也很重。

17.2 Prefill 推理

Prompt Prefill 一次有较多 Token：

Batch × Prompt Length

Expert GEMM 相对较大，GPU 利用率通常比 Decode 好；但路由不均仍可能造成 Rank 热点。

17.3 Decode 推理

每条序列每步只产生一个新 Token。若 Batch 不大：

- T 很小
- 每 Expert 收到的 Token 更少
- GEMM 极小
- All-to-All 延迟难摊薄

同时所有 Expert 权重仍需驻留在 GPU 集群内，容易出现：

- Memory Capacity 压力；
- Expert Weight 访问不连续；
- 跨 GPU 通信主导 TPOT；
- 某些热门 Experts 形成尾延迟；
- Expert Cache/Offload 命中率问题。

所以“MoE 训练吞吐高”不代表“MoE 单请求推理延迟低”。

十八、MoE 与 Dense 模型的核心对比

维度	Dense Transformer	Sparse MoE Transformer
FFN 参数	每层一套	每层多个 Routed/Shared Experts
每 Token 执行路径	固定	Router 动态选择
总参数与计算关系	基本同步增长	总参数按 E 增长，激活计算按 k 增长
通信	TP/DP/PP 等	额外 Token Dispatch/Combine All-to-All
GEMM Shape	相对规则	每 Expert Token 数动态，易产生小 GEMM
负载均衡	主要是静态切分	Router 动态负载，需均衡策略
显存	参数与优化器状态可预测	Expert 总参数很大，分片与优化器更复杂
容错/调试	路径固定，较容易	动态路由、Token Drop、通信和质量耦合
推理解码	Batch GEMM 相对稳定	小 Batch 时 Expert GEMM 与 A2A 难摊薄
优势	简单、稳定、硬件友好	大参数容量、条件计算、潜在专家专门化

十九、MoE 与各种并行策略的关系

MoE 不是替代其他并行策略，而是增加一个新的并行维度：Expert Parallel。

DP: 复制模型/分片训练数据
TP: 切单个 Dense/Expert 内部矩阵
PP/VPP: 沿层深度切模型
CP: 切序列上下文
EP: 把不同 Experts 放到不同 Rank

常见组合：

$TP \times PP/VPP \times DP/FSDP \times EP \times CP$

关键拓扑原则：

- TP 每层频繁通信，优先放高速 NVLink 域；
- EP All-to-All 对跨节点网络非常敏感；
- EP 太小，每卡 Expert 参数多、显存压力高；
- EP 太大，All-to-All 范围大、每 Expert Token 少、GEMM 更碎；
- DP/FSDP 如何处理 Expert 参数，需要区分 Dense 参数组和 Expert 参数组；
- 某些框架要求 EP 与 TP 组合时启用 Sequence Parallel；
- Attention 与 MoE 的最优并行布局可能不同，因此出现 Parallel Folding 等解耦映射。

二十、最小 PyTorch MoE 实现

下面代码是单 GPU、Dropless、Top-2 的教学实现。它保留真实 Router、Top-k、按 Expert 分组计算和加权合并，但没有分布式 All-to-All、容量限制和高性能 Grouped GEMM。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Expert(nn.Module):
    def __init__(self, hidden_size: int, ffn_size: int):
        super().__init__()
        self.w1 = nn.Linear(hidden_size, ffn_size, bias=False)
        self.w2 = nn.Linear(ffn_size, hidden_size, bias=False)

    def forward(self, x):
        # x: [tokens_for_this_expert, H]
        return self.w2(F.relu(self.w1(x)))
```

```

class SimpleTopKMoE(nn.Module):
    def __init__(
        self,
        hidden_size=8,
        ffn_size=16,
        num_experts=4,
        top_k=2,
    ):
        super().__init__()
        self.hidden_size = hidden_size
        self.num_experts = num_experts
        self.top_k = top_k

        self.router = nn.Linear(
            hidden_size,
            num_experts,
            bias=False,
        )

        self.experts = nn.ModuleList([
            Expert(hidden_size, ffn_size)
            for _ in range(num_experts)
        ])

    def forward(self, x):
        # x: [B, S, H]
        batch, seq, hidden = x.shape
        flat = x.reshape(-1, hidden)      # [T, H]
        tokens = flat.shape[0]

        # 1. Route
        logits = self.router(flat)        # [T, E]
        probs = torch.softmax(logits, dim=-1)

        top_weights, top_indices = torch.topk(
            probs,
            k=self.top_k,
            dim=-1,
        )                                # both [T, k]

        # 只在选中的 k 个 Experts 内重新归一化。
        top_weights = top_weights / top_weights.sum(
            dim=-1,
            keepdim=True,
        )

        # 2~4. Dispatch、Expert Compute、Combine
        output = torch.zeros_like(flat)   # [T, H]
        tokens_per_expert = []

        for expert_id, expert in enumerate(self.experts):
            # selected: [T, k], 表示每个 Token 的每个 Top-k Slot
            # 是否选择了当前 Expert。
            selected = top_indices == expert_id

            # 找出所有选择当前 Expert 的 Token 和对应 Top-k Slot。
            token_idx, slot_idx = torch.where(selected)
            tokens_per_expert.append(token_idx.numel())

            if token_idx.numel() == 0:
                continue

            expert_input = flat[token_idx]      # [T_e, H]
            expert_output = expert(expert_input) # [T_e, H]

            gate = top_weights[token_idx, slot_idx] # [T_e]
            weighted = expert_output * gate.unsqueeze(-1)

            # 多个 Expert 对同一个 Token 的贡献累加。
            output.index_add_(0, token_idx, weighted)

        output = output.reshape(batch, seq, hidden)

```

```

        aux = {
            "router_logits": logits,
            "router_probs": probs,
            "top_indices": top_indices,
            "top_weights": top_weights,
            "tokens_per_expert": torch.tensor(
                tokens_per_expert,
                device=x.device,
            ),
        }
        return output, aux

torch.manual_seed(0)

model = SimpleTopKMoE(
    hidden_size=8,
    ffn_size=16,
    num_experts=4,
    top_k=2,
)

x = torch.randn(2, 3, 8) # [B=2, S=3, H=8]
y, aux = model(x)

print("input shape:", x.shape)
print("output shape:", y.shape)
print("top indices shape:", aux["top_indices"].shape)
print("top weights shape:", aux["top_weights"].shape)
print("tokens per expert:", aux["tokens_per_expert"].tolist())
print("assignments:", int(aux["tokens_per_expert"].sum()))

# 最小反向验证: Router 和被选 Experts 都能收到梯度。
loss = y.square().mean()
loss.backward()

print("router grad exists:", model.router.weight.grad is not None)

```

预期 Shape:

```

input shape: [2,3,8]
output shape: [2,3,8]
top indices shape: [6,2]
top weights shape: [6,2]
assignments: 12

```

这段代码说明了什么

- Router 对每个 Token 产生 **[E]** Scores;
- Top-k 产生 **[T,k]** Expert IDs 和 Weights;
- 同一个 Token 会被多个 Experts 处理;
- **index_add_** 把多个 Expert 的加权输出合并回原 Token;
- Router 的 Top-k Index 是离散选择, 但选中权重仍可对 Router 反向传播;
- 未被选中的 Expert 本批可能没有梯度。

它没有说明什么

生产实现还需要：

- Expert Capacity/Token Drop;
- EP All-to-All;
- 高性能 Permute/Unpermute;
- Grouped GEMM;
- Router Aux Loss/Z-Loss;
- Shared Experts;
- FP8/BF16;
- TP/DP/PP/CP 组合;
- 通信与计算 Overlap;
- 分布式 Checkpoint;
- 动态 Shape 与 CUDA Graph。

二十一、训练中应该监控哪些指标

Router/负载

```
Tokens per Expert Histogram
Max / Mean Tokens per Expert
Expert Utilization
Router Entropy
Top-k Probability
Aux Loss
Z-Loss
Drop Rate / Overflow Rate
```

负载不均衡比：

```
Imbalance Ratio = max_e(T_e) ÷ mean_e(T_e)
```

越接近 1 越均匀，但也不能只追求绝对均匀而牺牲语义路由。

通信

```
Dispatch All-to-All Time
Combine All-to-All Time
```

Exposed A2A Time
Per-Rank Send/Recv Bytes
Cross-Node Traffic
Permutation/Unpermutation Time

Expert Compute

Grouped GEMM Time
Per-Expert GEMM M Dimension
Tensor Core Utilization
Kernel Launch Count
Expert Compute Imbalance

端到端

Step Time
Tokens/s
MFU
Peak Memory
Optimizer Memory
Activation Memory
Checkpoint Time

不要只看“总 FLOP 少了多少”。MoE 的性能可能被通信、Permutation 和小 GEMM 完全主导。

二十二、常见失败模式

失败 1：Expert Collapse

少数 Experts 获得大部分 Token，其他 Experts 不训练。

排查：Tokens-per-Expert、Router Entropy、Aux Coeff、Input Distribution。

失败 2：Capacity 太小，Drop Rate 高

模型质量下降，部分 Token 路径丢失。

排查：Capacity Factor、Top-k、Router Skew、Drop Policy。

失败 3：Capacity 太大，Padding 浪费

固定 Shape 方便，但大量 Padding 吃掉计算和通信。

失败 4: EP Group 跨慢网络

All-to-All 暴露时间过高，训练吞吐下降。

失败 5: Experts 太细，Grouped GEMM 仍很小

模型表达更灵活，但 GPU Tensor Core 吃不饱。

失败 6: Aux Loss 太强

路由很均匀，但专家无法形成有意义的专门化。

失败 7: 推理 Batch 太小

每 Expert 只有少量 Token，Decode TPOT 被通信和 Kernel Launch 主导。

失败 8: 只优化通信，不看 HBM 与 Permutation

All-to-All 变快后，Permutation、Unpermutation 或 Expert GEMM 成为新瓶颈。

二十三、什么时候适合使用 MoE

适合：

- 希望在相近每 Token FLOP 下扩大参数容量；
- 训练 Token 规模足够大，能让 Experts 获得充足数据；
- 集群有高速互联，能承受 EP All-to-All；
- 有成熟的 Router、Grouped GEMM、Dispatcher 和并行框架；
- 目标是大规模预训练/高吞吐 Serving；
- 可以接受更高工程复杂度和参数显存。

不适合：

- 模型规模不大，Dense 已能满足容量需求；
- 单请求低延迟是绝对优先，Batch 很小；
- 跨节点网络差；
- 训练数据不足，Experts 难以专门化；

- 团队缺少路由、通信、Checkpoint 和可观测性能力；
- 显存只能容纳 Active 参数，却无法容纳/分片全部 Expert 参数。

二十四、常见误区

误区 1: MoE 只加载被选中的 Experts，所以总参数不占显存

错误。训练时全部 Expert 参数及其梯度/优化器状态必须存在于某些设备或存储层中；EP 只是把它们分散，不是让未选 Experts 消失。

误区 2: Top-2 表示同时运行两个完整模型

错误。通常只是同一 Transformer Block 的两个 Expert FFN，被选结果在该层加权合并。

误区 3: MoE 总参数增加 E 倍，计算一定不变

错误。每 Token 计算取决于 $k \times F_{\text{expert}}$ 、Router、Shared Experts、Padding、通信和实现开销。

误区 4: 负载越均匀，模型质量一定越好

错误。系统需要避免严重热点，但过强均衡可能损害专家专门化。

误区 5: All-to-All 是唯一瓶颈

错误。Permutation、动态小 GEMM、内存、Router、Checkpoint 和 Decode 小 Batch 都可能成为主要瓶颈。

误区 6: MoE 更省训练显存

不一定。Active FLOP 较少不代表总参数、梯度和优化器状态少；MoE 的参数显存常显著高于相近计算量的 Dense 模型。

二十五、关键总结

MoE 的完整因果链：

```
Dense FFN 只能通过增大同一 MLP 扩大参数
→ 每 Token 计算与参数一起增长

MoE 放置 E 个 Experts
→ Router 为每 Token 选择 k 个,  $k \ll E$ 
→ 总参数由 E 决定, 激活计算由 k 决定
→ 参数容量与每 Token 计算解耦

但动态路由产生不均匀 Token Assignment
→ 需要负载均衡与容量策略
→ Experts 分布到多 GPU 后需要两次 All-to-All
→ 形成通信、动态小 GEMM、显存和可观测性难题
```

分析任何 MoE 系统时固定问六个问题：

1. **Architecture**：多少 Experts? Top-k? Expert 宽度? 是否 Shared Experts?
2. **Routing**：Softmax/Sigmoid? Aux Loss? Capacity? Dropless?
3. **Compute**：每 Expert 收到多少 Token? Grouped GEMM 是否足够大?
4. **Communication**：EP 多大? All-to-All 是否跨节点? 暴露时间多少?
5. **Memory**：全部 Expert 参数、梯度、优化器状态如何分布?
6. **Serving**：Prefill/Decode Batch 多大? 专家权重和通信能否摊薄?

可选自测

1. Shape 与容量题

```
B=4, S=2048, E=64, k=2, CapacityFactor=1.25
```

请计算：

- T ；
- 总 Assignments；
- 每 Expert 平均 Assignments；
- Capacity C 。

2. 参数与 FLOP 题

一个 Dense FFN 使用 $H=4096$, $F_{\text{dense}}=14336$; MoE 使用 $E=8$, $k=2$ 。如果希望主 Expert FLOP 近似 Dense FFN, 应如何估算 F_{expert} ? MoE 全部 Expert 参数约是 Dense FFN 的多少倍?

3. 工程决策题

某 MoE 训练中:

```
All-to-All = 35% Step Time  
Grouped GEMM = 25%  
Permutation = 15%  
Max/Mean Expert Tokens = 2.8  
Drop Rate = 4%
```

你会按什么顺序优化? 哪些优化是在减少通信量, 哪些是在减少暴露时间, 哪些是在改善负载与计算效率?

继续学习

1. 前置: MoE Router 与负载均衡损失 —— 深入 Top-k、Aux Loss、Z-Loss 和 Aux-Loss-Free Balancing。
2. 同层对比: Dense FFN、Switch、Mixtral、DeepSeekMoE —— 对比 Top-1/Top-2、Shared Experts 与细粒度专家。
3. 下游应用: Expert Parallel 与 DeepEP —— 深入 All-to-All Dispatcher、通信 Overlap 和跨节点拓扑。

回复 1、2 或 3 继续; 也可以说“稍后”或“不感兴趣”。

事实来源与版本边界

本文的稳定原理基于 Sparsely-Gated MoE、GShard、Switch Transformer 与 DeepSeekMoE 的公开论文, 以及截至 2026-07-29 的 NVIDIA Megatron Core MoE 官方文档。不同模型与框架在 Router Score、Top-k 归一化、Capacity、Token Drop、Aux Loss、Shared Expert、Dispatcher 和并行映射上存在差异; 具体配置、函数名和性能应以目标版本源码与实测为准。